
Lab 4: Microsoft Fabric End-to-End Optimization & Governance Lab

This version includes:

- Exact UI actions
 - What to click
 - What to enter
 - Expected output after every step
 - Validation checkpoints
 - Common issues
 - Enterprise best practices
-

LAB OVERVIEW

Goal

You will build:

CSV Files



Lakehouse



Pipeline



Warehouse



Governance



Access Controls



Lineage Tracking

Step 1 — Verify Workspace Configuration - Open Existing Workspace

Actions

From left panel:

Workspaces

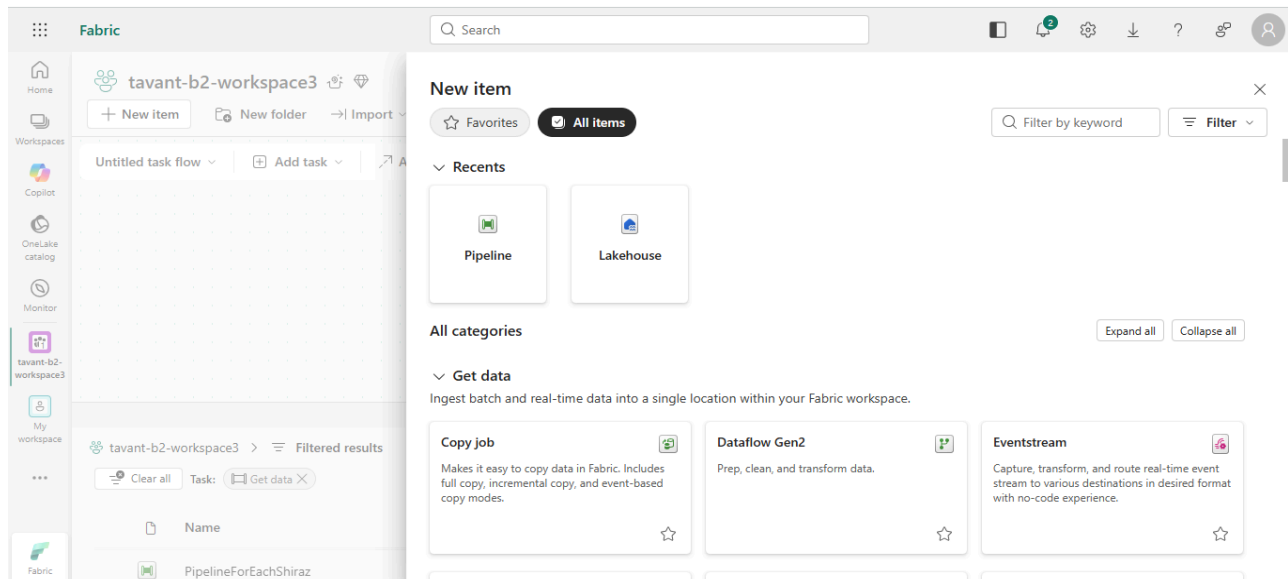
Open your workspace:

tavant-b2-workspace3

Expected Output

You should see:

Item	Expected
New Item button	Visible
Lakehouse option	Available
Data Pipeline	Available
Warehouse	Available



Step 2 — Create Lakehouse

Actions

Click:

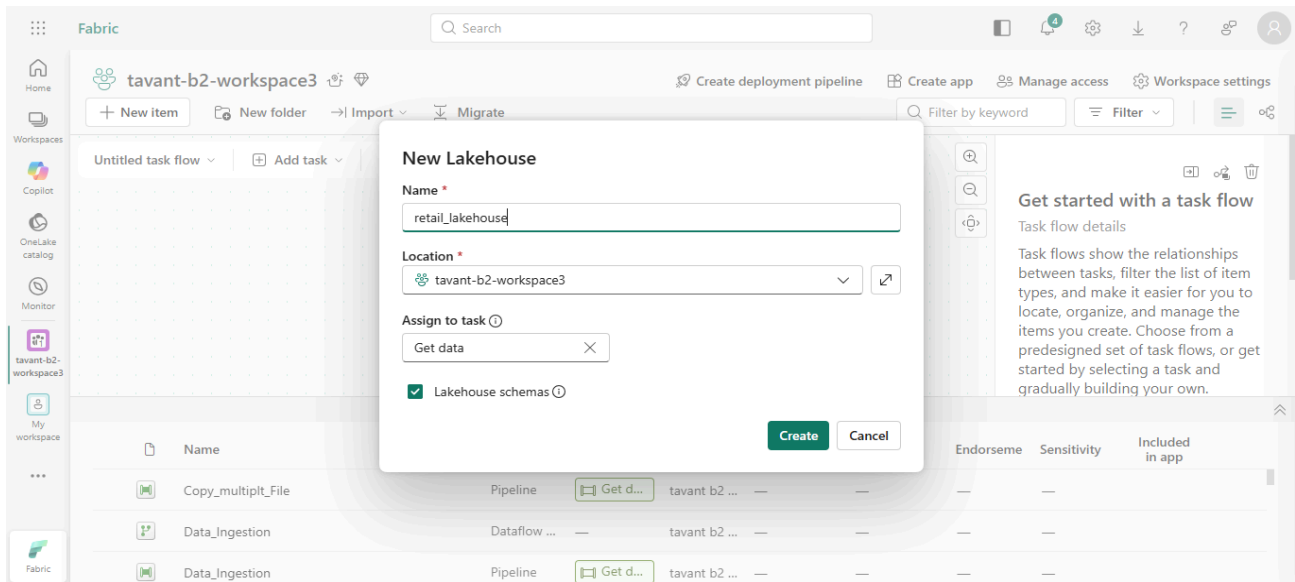
New Item → Lakehouse

Fill:

Field	Value
Name	retail_lakehouse
Lakehouse schemas	Checked

Click:

Create



Expected Output

Lakehouse opens successfully.

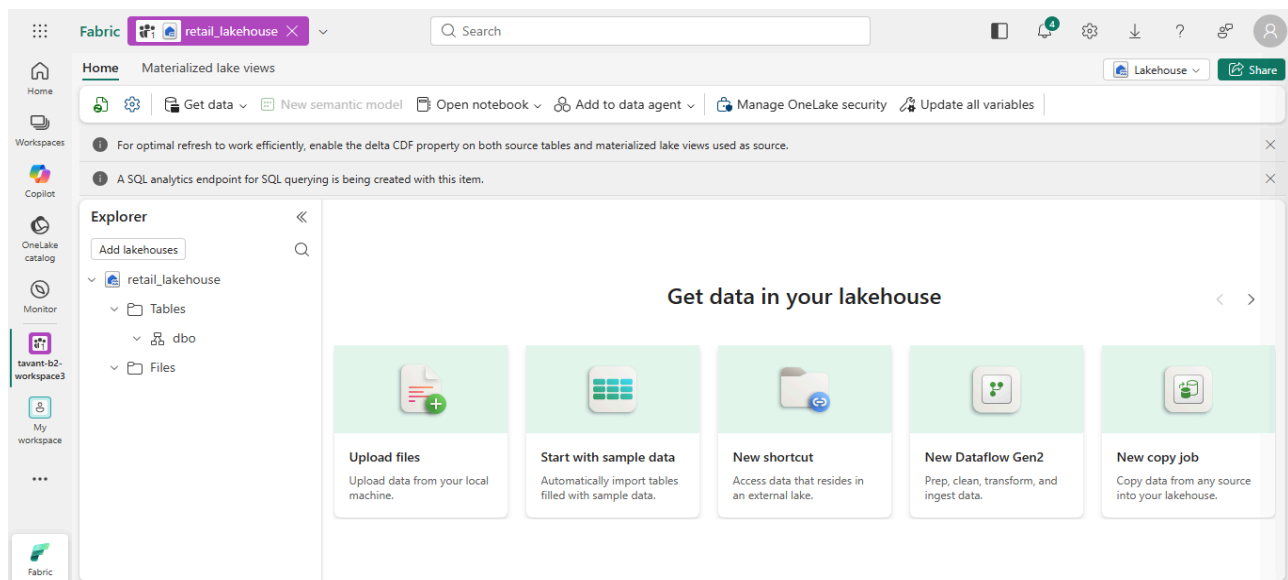
You should see:

Files

Tables

SQL analytics endpoint

Default semantic model



Step 3 — Upload Files

Actions

Inside Lakehouse:

Files → Upload → Upload Files

Upload:

- customers.csv
- products.csv
- sales.csv

Expected Output

Files appear under:

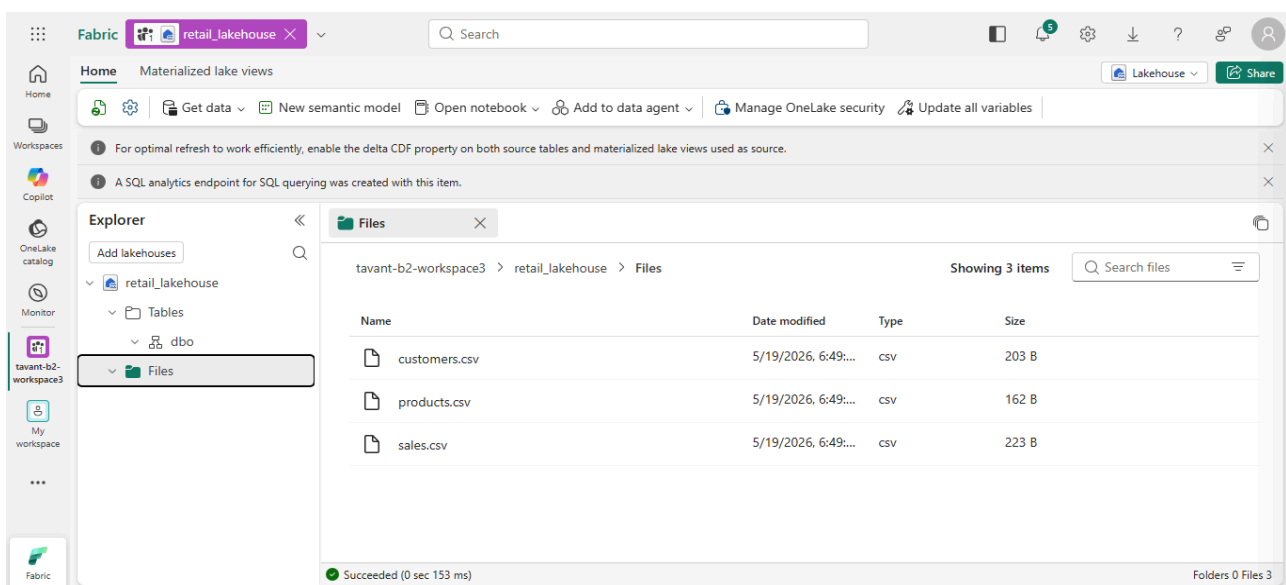
Files

Example:

customers.csv

products.csv

sales.csv



Step 4 — Create Data Pipeline

Actions

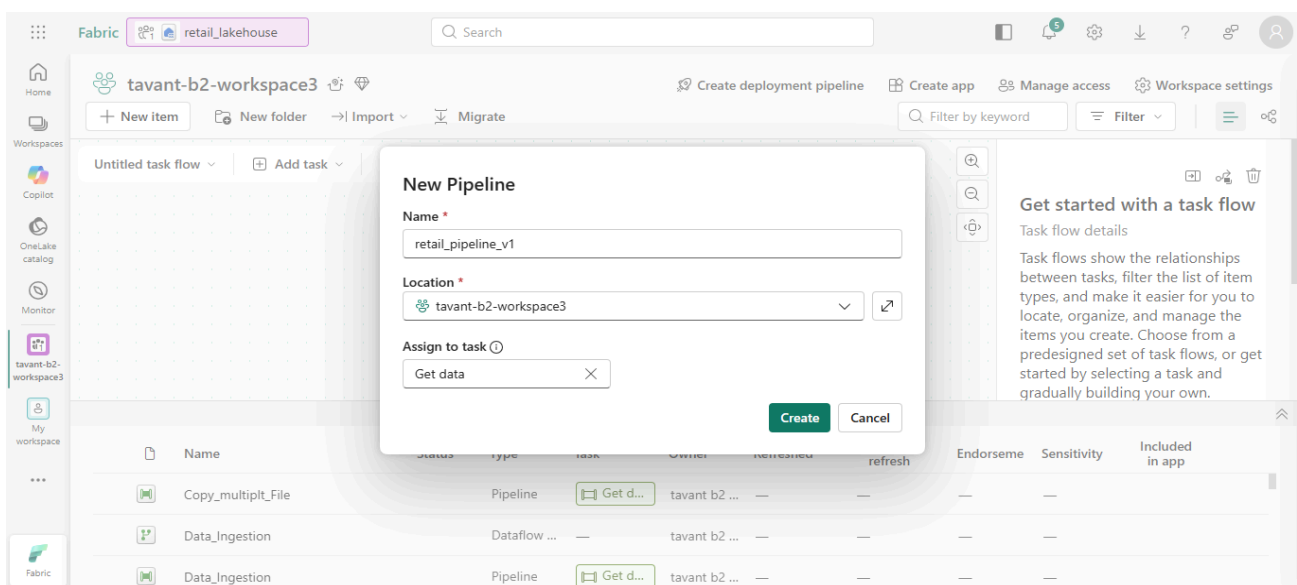
Click:

New Item → Data Pipeline

Name:

retail_pipeline_v1

Click Create.



Expected Output

Pipeline canvas opens.

Top ribbon shows:

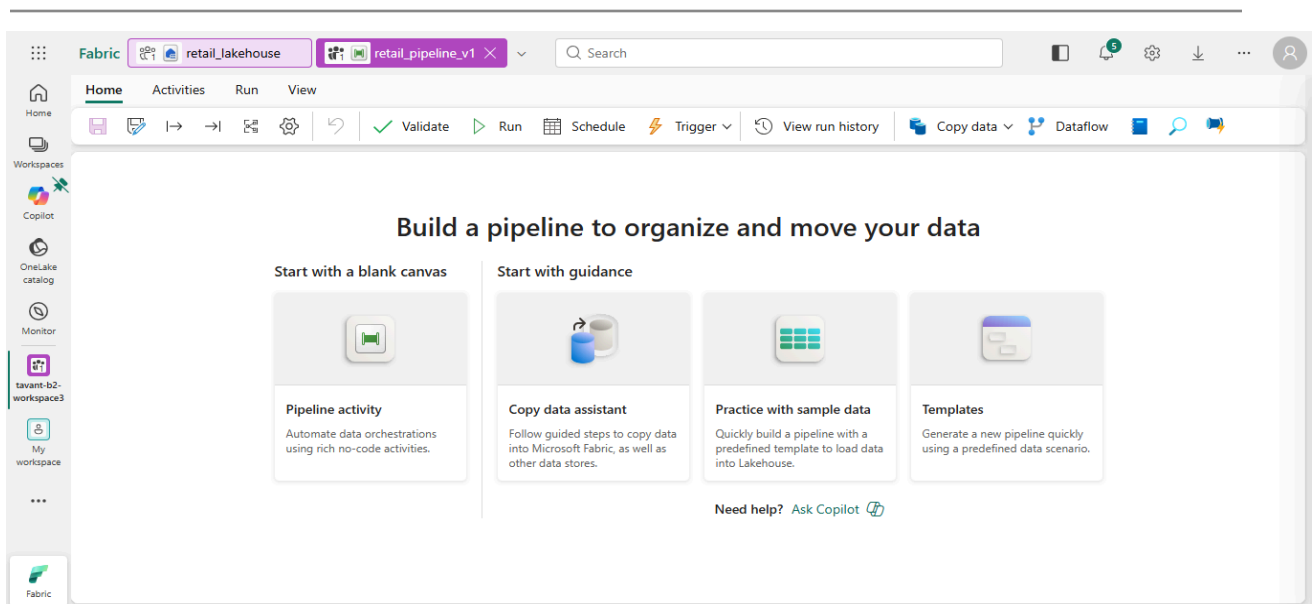
Copy data

Dataflow

Notebook

Lookup

Get metadata



Step 5 — Drag Copy Data Activity

Actions

Click:

Copy data

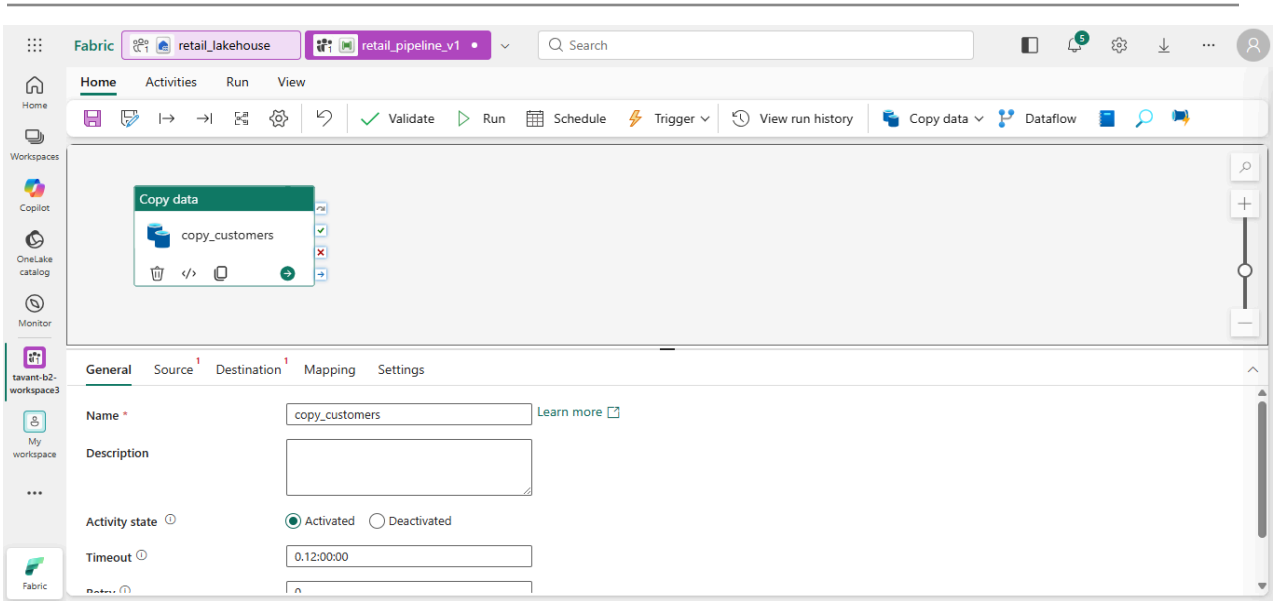
Activity appears on canvas.

Rename activity:

copy_customers

Expected Output

Copy activity visible on pipeline canvas.



Step 6 — Configure Source

Actions

Open:

Source tab

Fill:

Field	Value
Connection	retail_lakehouse

Root Folder	Files
File Path Type	File path

Click:

Browse

Select:

customers.csv

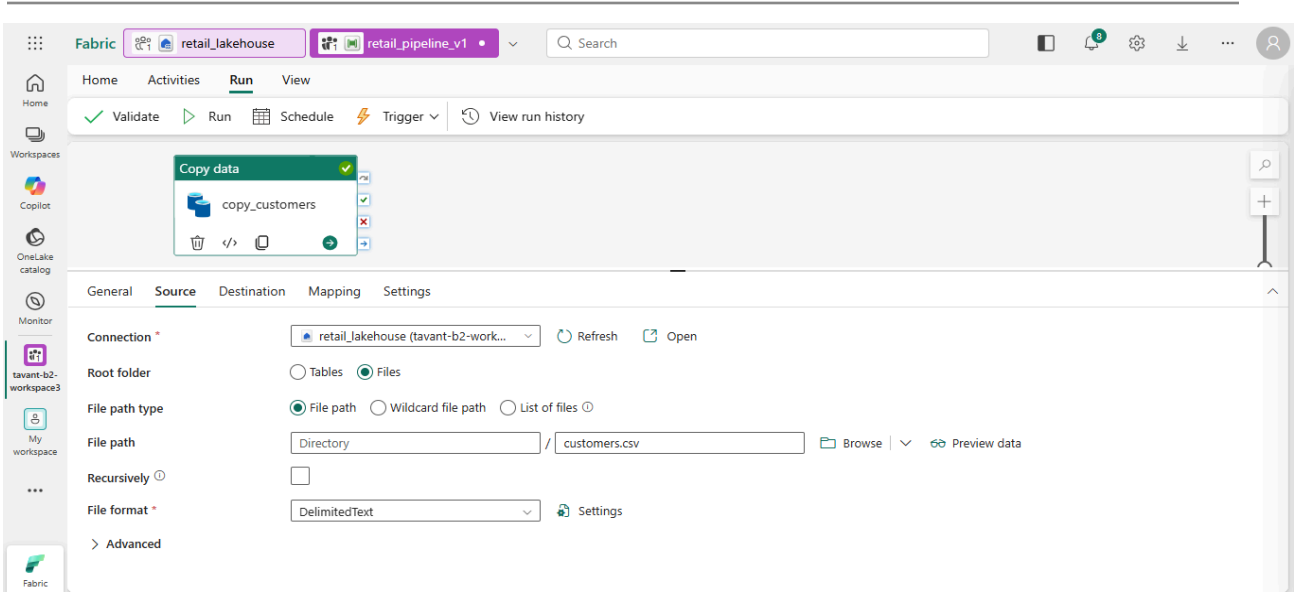
Change File Format

Change:

Binary

TO:

DelimitedText

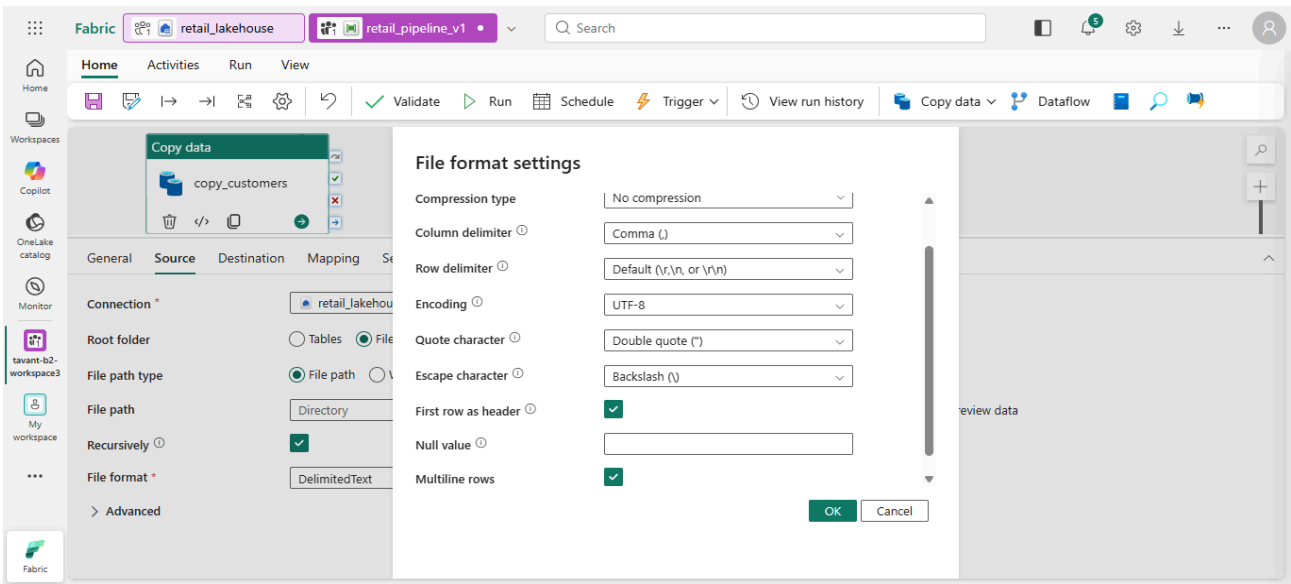


Click:

Settings

Fill:

Setting	Value
Column delimiter	Comma
First row as header	TRUE
Encoding	UTF-8



Click:

Preview data

Expected Output

Preview table appears:

customer_id	customer_name	city
101	Rahul Sharma	Mumbai

Actions

Destination tab

Field	Value
Connection	retail_lakehouse
Root Folder	Tables
Schema name	dbo

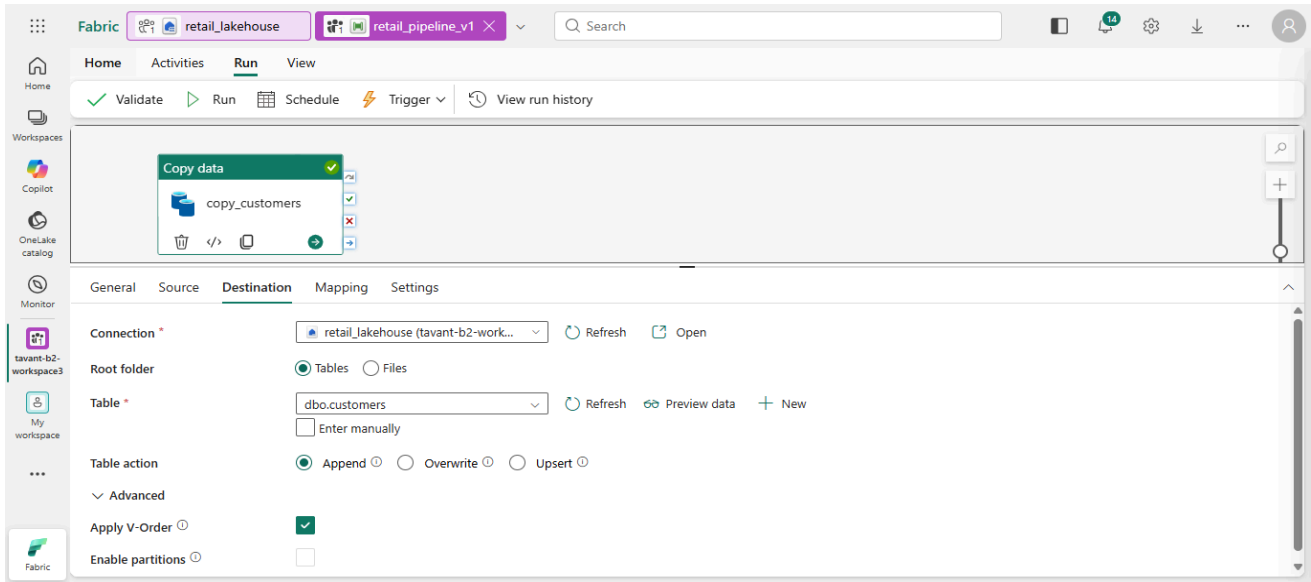
Table	customers
Table Action	Append

Check:

Apply V-Order

Leave unchecked:

Enable partitions



Expected Output

Destination configured as:

dbo.customers

Step 8 — Configure Mapping

Actions

Open:

Mapping tab

Click:

Import Schemas

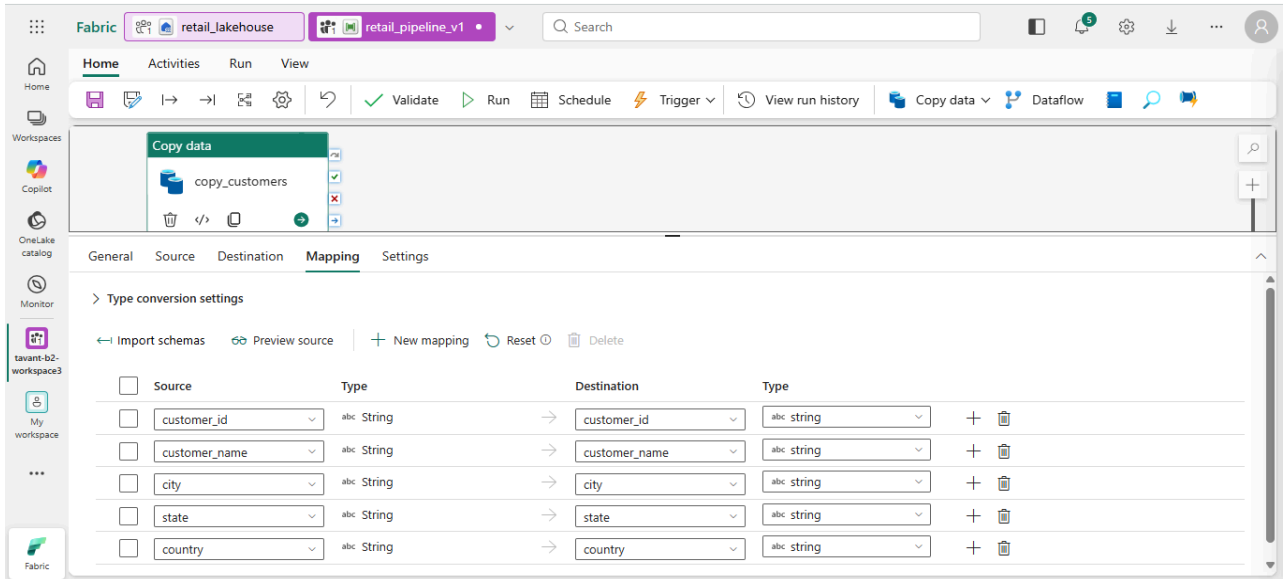
Expected Output

Column mapping appears.

Example:

Source	Destination
customer_id	customer_id

customer_name	customer_name
---------------	---------------



Step 9 — Configure Pipeline Settings

Actions

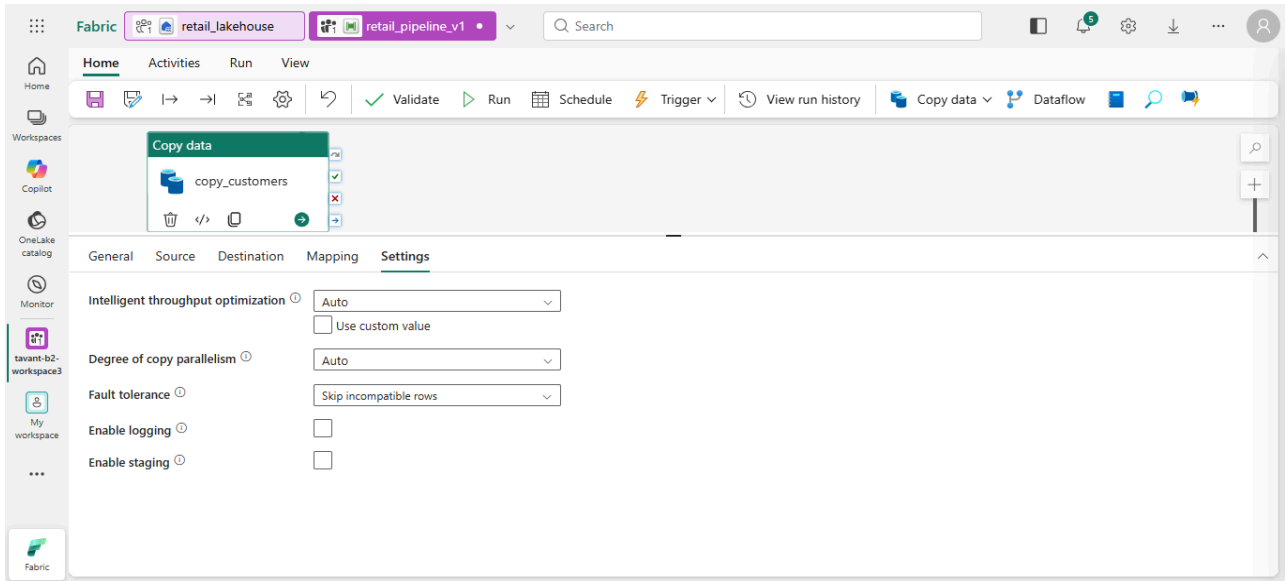
Open:

Settings tab

Fill:

Setting	Value
---------	-------

Intelligent throughput optimization	Auto
Degree of copy parallelism	Auto
Fault tolerance	Skip incompatible rows
Enable logging	OFF



Expected Output

Settings saved successfully.

Step 10 — Execute Pipeline

Actions

Click:

Run

OR

Run pipeline

Expected Output

Pipeline status changes:

Queued

→ In Progress

→ Succeeded

Green success icon appears.

The screenshot displays the Microsoft Fabric user interface. At the top, the breadcrumb navigation shows 'retail_lakehouse' and 'retail_pipeline_v1'. The 'Run' tab is active, showing a 'Copy data' activity with a green checkmark and a 'copy_customers' output. Below the activity, the 'Output' tab is selected, displaying the 'Pipeline run' details. The pipeline run ID is '22c1531b-6ffc-46fc-9852-4351d18583' and the status is 'Succeeded'. A table below shows the activity details:

Activity name	Activity status	Run start	Duration	Input	Output
copy_customers	Succeeded	5/19/2026, 7:12:55 PM	21s		

Step 11 — Validate Lakehouse Tables

Actions

Return to:

retail_lakehouse

Open:

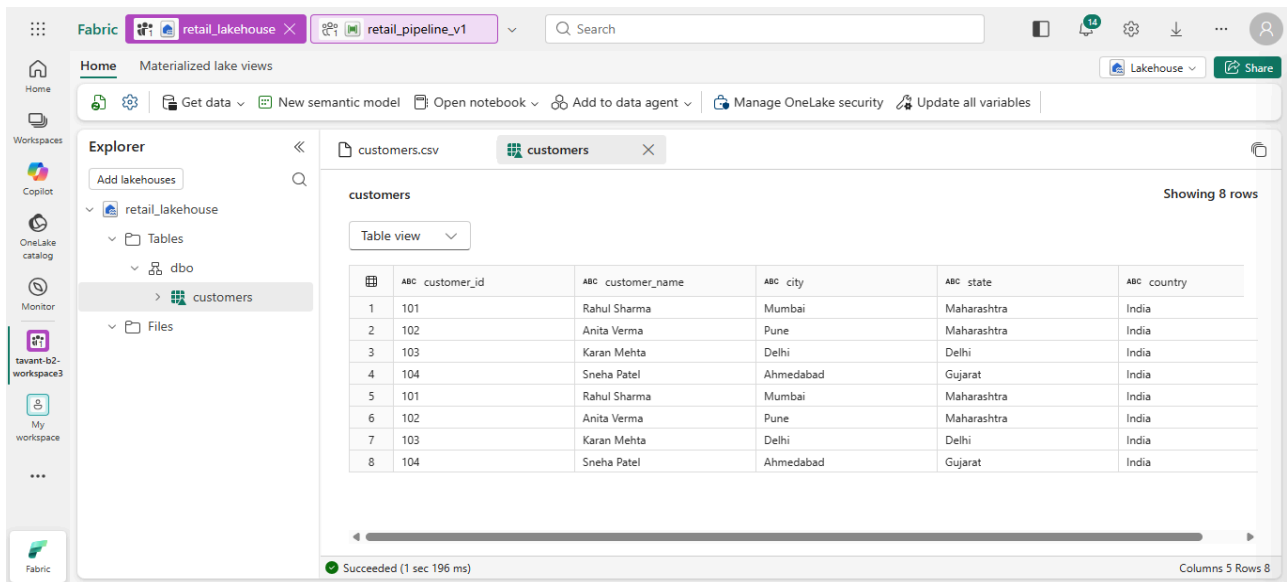
Tables

Expected Output

You should see:

customers

Open table preview.



Step 12 — Repeat for Remaining Files

Repeat Steps 5–11 for:

CSV	Table
products.csv	products
sales.csv	sales

Expected Output

Tables visible:

customers

products

sales

Fabric

retail_lakehouse

retail_pipeline_v1

Search

Home

Activities

Run

View

Validate

Run

Schedule

Trigger

View run history

Workspaces

Copilot

OneLake catalog

Monitor

tavant-b2-workspace3

My workspace

...

Fabric

Copy data

copy_customers

Copy data

copy_products

Parameters

Variables

Settings

Output

Library variables

Pipeline run ID

b61906df-6f93-47ab-abd1-124f12a683bf

View run detail

Pipeline status

Succeeded

Export to CSV

Filter

Filter by keyword

Showing 1 - 2 items

Activity name	Activity status	Run start	Duration	Input	Output
copy_products	Succeeded	5/19/2026, 7:51:51 PM	20s		
copy_customers	Succeeded	5/19/2026, 7:51:51 PM	19s		

Run Succeeded

Successfully ran retail_pipeline_v1 (Pipeline).

Copy data

copy_customers

Copy data

copy_products

Copy data

copy_sales

Parameters

Variables

Settings

Output

Library variables

Pipeline run ID

4a1c9857-1978-4ed1-9d5e-a4a265c61b58

View run detail

Pipeline status

Succeeded

Export to CSV

Filter

Filter by keyword

Showing 1 - 3 items

Step 13 — Add Notebook

Actions

Inside pipeline:

Click:

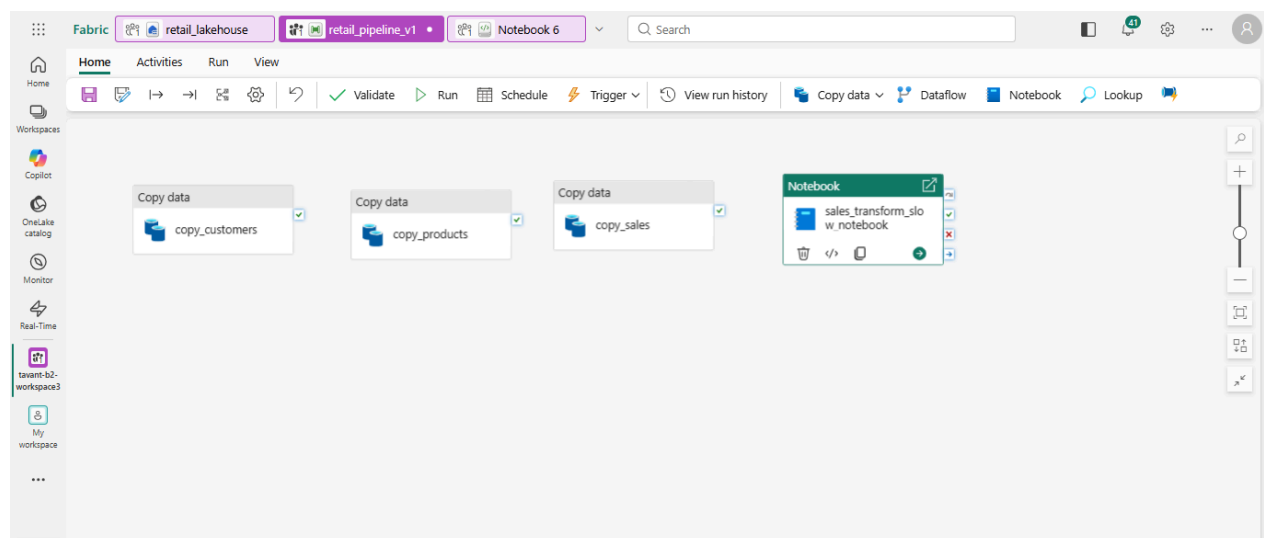
Notebook

Rename:

sales_transform_slow_notebook

Expected Output

Notebook activity added.



Step 14 — Add Inefficient Queries

Perform:

Multiple Sorts:

Run:

```
SELECT *
```

```
FROM dbo.sales
```

```
ORDER BY sale_amount ASC;
```

The screenshot shows the Microsoft Fabric Notebook 6 interface. The query editor contains the following SQL code:

```
1 SELECT *
2 FROM dbo.sales
3 ORDER BY sale_amount ASC;
```

The query has been executed successfully, as indicated by the status bar: "31 sec - Session ready in 9 sec 353 ms. Command executed in 21 sec 981 ms by Chetna Chaudhary on 5/19/2026, 10:27:30 PM".

The results are displayed in a table view with 7 columns and 36 rows. The columns are: `abc sale_id`, `abc customer_id`, `abc product_id`, `abc quantity`, `abc sale_amount`, `abc sale_date`, and `abc region`. The first 9 rows of the table are shown below:

	abc sale_id	abc customer_id	abc product_id	abc quantity	abc sale_amount	abc sale_date	abc region
1	S003	103	P104	1	15000	2026-05-03	North
2	S003	103	P104	1	15000	2026-05-03	North
3	S003	103	P104	1	15000	2026-05-03	North
4	S003	103	P104	1	15000	2026-05-03	North
5	S003	103	P104	1	15000	2026-05-03	North
6	S003	103	P104	1	15000	2026-05-03	North
7	S003	103	P104	1	15000	2026-05-03	North
8	S003	103	P104	1	15000	2026-05-03	North
9	S003	103	P104	1	15000	2026-05-03	North

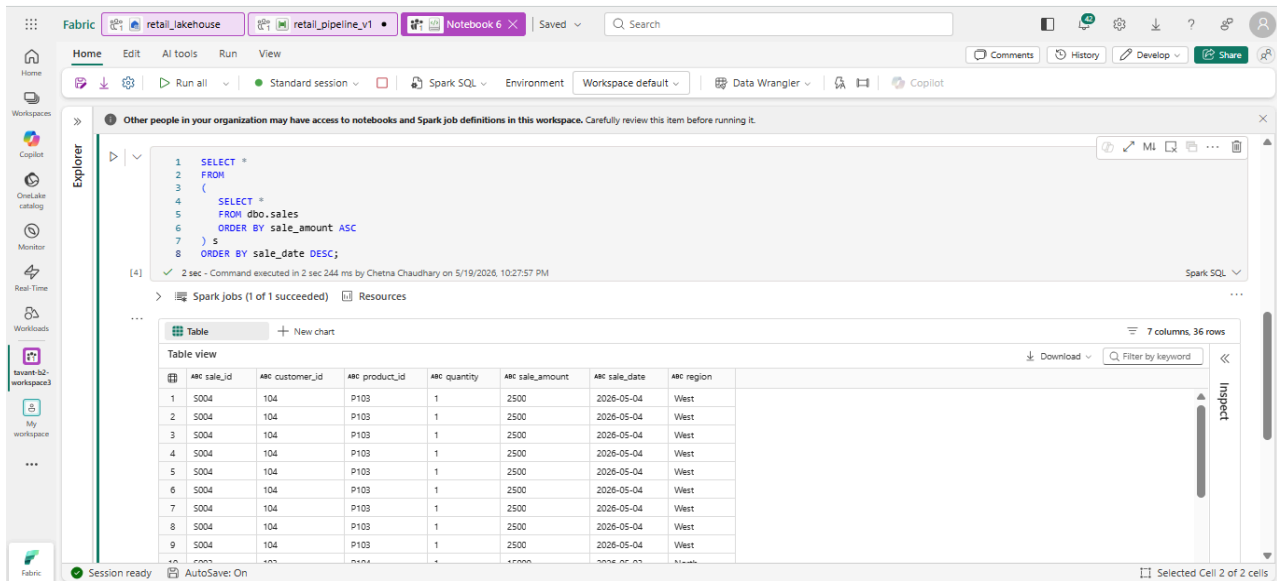
Run Another Sort:

```
SELECT *
```

```
FROM
```



```
(  
  
SELECT *  
  
FROM dbo.sales  
  
ORDER BY sale_amount ASC  
  
) s  
  
ORDER BY sale_date DESC;
```



Expected Output

Rows returned sorted twice.

Example:

sale_id	sale_amount	sale_date
---------	-------------	-----------

S001	75000	2026-05-01
------	-------	------------

Duplicate Filtering:

Run:

```
SELECT DISTINCT *
```

```
FROM dbo.sales;
```

```
1 SELECT DISTINCT *
2 FROM dbo.sales;
```

✓ 15 sec - Command executed in 15 sec 94 ms by Chetna Chaudhary on 5/19/2026, 10:29:57 PM

> Spark jobs (2 of 2 succeeded) Resources Log

Table

New chart

Table view

	ABC sale_id	ABC customer_id	ABC product_id	ABC quantity	ABC sale_amount	ABC sale_date	ABC region	
1	S001	101	P101	1	75000	2026-05-01	West	
2	S004	104	P103	1	2500	2026-05-04	West	
3	S002	102	P102	2	2400	2026-05-02	West	
4	S003	103	P104	1	15000	2026-05-03	North	

More Expensive Deduplication:

```
SELECT DISTINCT
```

```
sale_id,
```

```
customer_id,
```

product_id,

quantity,

sale_amount,

sale_date,

region

FROM dbo.sales;

```
1  SELECT DISTINCT
2  sale_id,
3  customer_id,
4  product_id,
5  quantity,
6  sale_amount,
7  sale_date,
8  region
9  FROM dbo.sales;
```

✓ 3 sec - Command executed in 3 sec 224 ms by Chetna Chaudhary on 5/19/2026, 10:30:26 PM

>  Spark jobs (2 of 2 succeeded)  Resources  Log

 Table  New chart

Table view

	ABC sale_id	ABC customer_id	ABC product_id	ABC quantity	ABC sale_amount	ABC sale_date	ABC region	
1	S001	101	P101	1	75000	2026-05-01	West	
2	S004	104	P103	1	2500	2026-05-04	West	
3	S002	102	P102	2	2400	2026-05-02	West	
4	S003	103	P104	1	15000	2026-05-03	North	

Expected Output

Unique rows returned.

Repeated Joins:

Run:

```
SELECT
```

```
s.sale_id,
```

```
s.sale_amount,
```

```
c.customer_name,
```

```
c.city
```

```
FROM dbo.sales s
```

```
INNER JOIN dbo.customers c
```

```
ON s.customer_id = c.customer_id;
```

```
1  SELECT
2  s.sale_id,
3  s.sale_amount,
4  c.customer_name,
5  c.city
6  FROM dbo.sales s
7  INNER JOIN dbo.customers c
8  ON s.customer_id = c.customer_id;
```

✓ 13 sec - Command executed in 13 sec 501 ms by Chetna Chaudhary on 5/19/2026, 10:31:17 PM

>  Spark jobs (11 of 11 succeeded)  Resources  Log

 Table

+ New chart

Table view

	ABC sale_id	ABC sale_amount	ABC customer_name	ABC city	
1	S001	75000	Rahul Sharma	Mumbai	
2	S001	75000	Rahul Sharma	Mumbai	
3	S001	75000	Rahul Sharma	Mumbai	
4	S001	75000	Rahul Sharma	Mumbai	
5	S001	75000	Rahul Sharma	Mumbai	
6	S001	75000	Rahul Sharma	Mumbai	
7	S001	75000	Rahul Sharma	Mumbai	
8	S001	75000	Rahul Sharma	Mumbai	
9	S001	75000	Rahul Sharma	Mumbai	
10	S002	2400	Anita Verma	Pune	

Add Another Join:

```
SELECT  
  
s.sale_id,  
  
s.sale_amount,  
  
c.customer_name,  
  
p.product_name,  
  
p.category  
  
FROM dbo.sales s  
  
INNER JOIN dbo.customers c  
  
ON s.customer_id = c.customer_id  
  
INNER JOIN dbo.products p  
  
ON s.product_id = p.product_id;
```

```

1  SELECT
2  s.sale_id,
3  s.sale_amount,
4  c.customer_name,
5  p.product_name,
6  p.category
7  FROM dbo.sales s
8  INNER JOIN dbo.customers c
9  ON s.customer_id = c.customer_id
10 INNER JOIN dbo.products p
11 ON s.product_id = p.product_id;

```

✓ 9 sec - Command executed in 9 sec 390 ms by Chetna Chaudhary on 5/19/2026, 10:31:55 PM

>  Spark jobs (10 of 10 succeeded)  Resources

Table + New chart						
Table view						
	ABC sale_id	ABC sale_amount	ABC customer_name	ABC product_name	ABC category	
1	S001	75000	Rahul Sharma	Laptop	Electronics	
2	S001	75000	Rahul Sharma	Laptop	Electronics	
3	S001	75000	Rahul Sharma	Laptop	Electronics	
4	S001	75000	Rahul Sharma	Laptop	Electronics	
5	S001	75000	Rahul Sharma	Laptop	Electronics	
6	S001	75000	Rahul Sharma	Laptop	Electronics	
7	S001	75000	Rahul Sharma	Laptop	Electronics	
8	S001	75000	Rahul Sharma	Laptop	Electronics	

Even More Inefficient Version:

SELECT *

FROM dbo.sales s

INNER JOIN dbo.customers c

ON s.customer_id = c.customer_id

INNER JOIN dbo.products p

ON s.product_id = p.product_id

ORDER BY s.sale_amount DESC;

```

1 SELECT *
2 FROM dbo.sales s
3 INNER JOIN dbo.customers c
4 ON s.customer_id = c.customer_id
5 INNER JOIN dbo.products p
6 ON s.product_id = p.product_id
7 ORDER BY s.sale_amount DESC;

```

✓ 5 sec - Command executed in 5 sec 830 ms by Chetna Chaudhary on 5/19/2026, 10:32:26 PM Spark SQL

> Spark Jobs (3 of 3 succeeded) Resources

Table + New chart 16 columns, 1000 rows

Table view Download Filter by keyword

	ABC sale_id	ABC customer_id	ABC product_id	ABC quantity	ABC sale_amount	ABC sale_date	ABC region	ABC customer_id	ABC customer_name	ABC city	ABC state	ABC country	ABC product_id
1	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
2	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
3	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
4	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
5	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
6	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
7	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
8	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
9	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
10	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101

Inspect

Expected Output

Joined dataset returned:

sale_id	customer_name	product_name
S001	Rahul Sharma	Laptop

Step 15 — Open Monitor Hub & Monitor Slow Pipeline

Actions

From left panel:

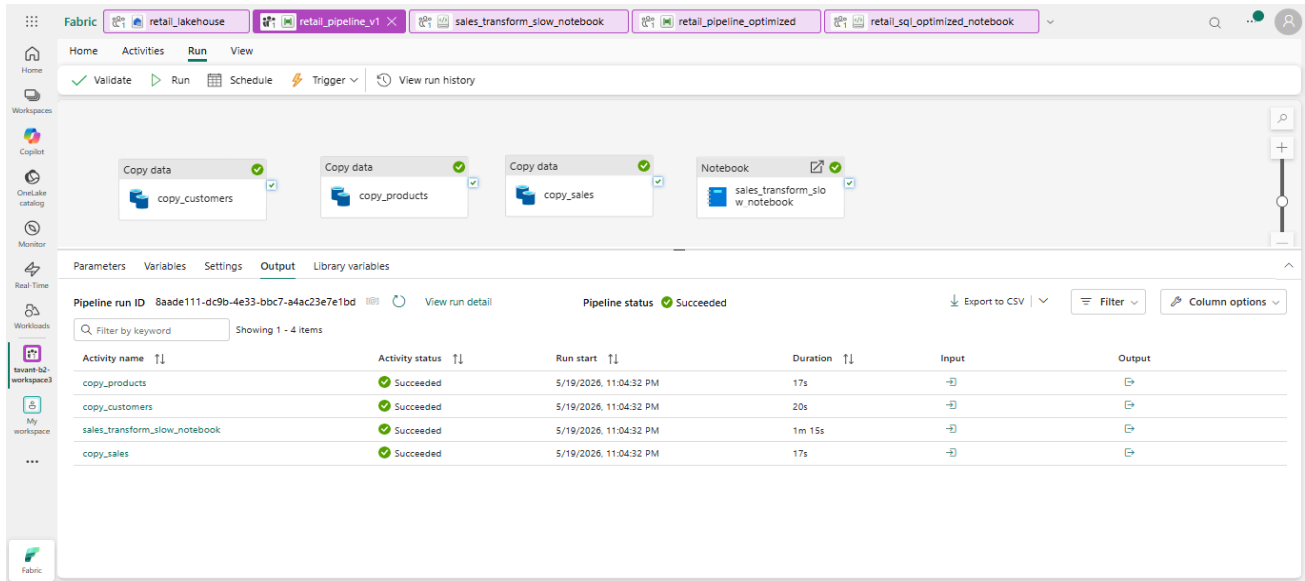
Monitor

Open pipeline run.

Expected Output

You should see:

Metric	Example
Runtime	Higher
Queue time	Increased
Copy duration	Slow



Step 16 — Duplicate Pipeline

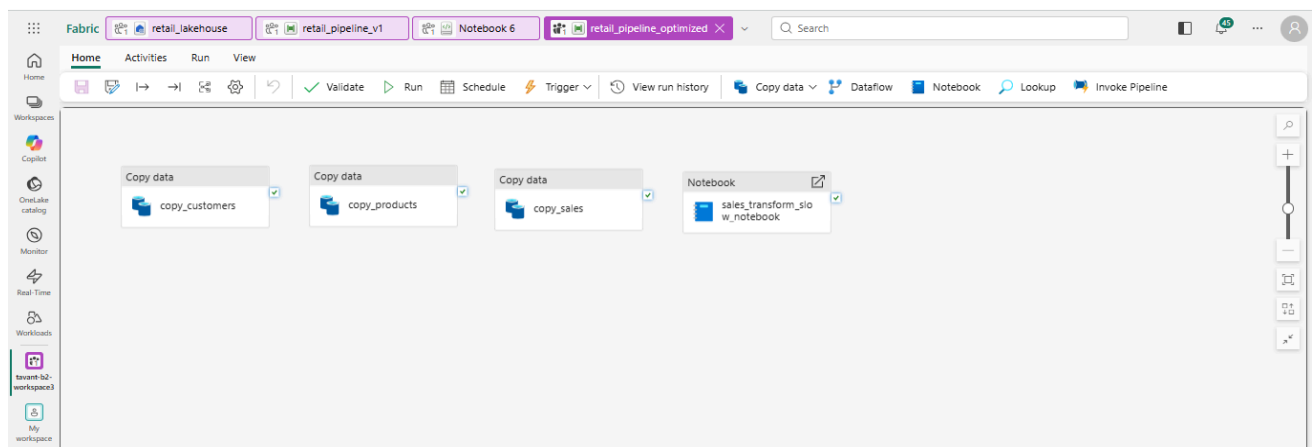
Actions

Duplicate:

retail_pipeline_v1

Rename:

retail_pipeline_optimized



Expected Output

Optimized copy created.

Step 17 — Improve Performance

Actions

Inside workspace locate:

sales_transform_slow_notebook

Click:

⋮ → Duplicate

Rename:

retail_sql_optimized_notebook

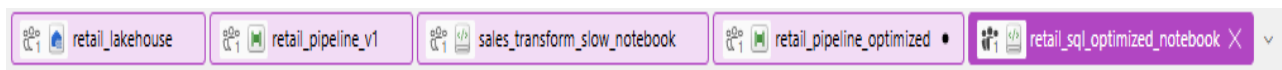
Set:

Parallel Copies = 4

Expected Output

You now have:

Notebook
sales_transform_slow_notebook
retail_sql_optimized_notebook



Open Optimized Notebook:

Actions

Open:

retail_sql_optimized_notebook

Expected Output

Notebook editor opens successfully.

Remove Repeated Sorts:

OLD SLOW QUERY

Remove this:

```
SELECT *  
FROM  
(  
    SELECT *  
    FROM dbo.sales  
    ORDER BY sale_amount ASC  
) s  
ORDER BY sale_date DESC;
```

REPLACE WITH OPTIMIZED QUERY:

```
SELECT
sale_id,
customer_id,
sale_amount,
sale_date
FROM dbo.sales;
```

```
1  SELECT
2  sale_id,
3  customer_id,
4  sale_amount,
5  sale_date
6  FROM dbo.sales;
```

✓ 3 sec - Command executed in 3 sec 178 ms by Chetna Chaudhary on 5/19/2026, 10:53:59 PM

> Spark jobs (3 of 3 succeeded)  Resources

 Table  New chart

Table view

	ABC sale_id	ABC customer_id	ABC sale_amount	ABC sale_date	
1	S001	101	75000	2026-05-01	
2	S002	102	2400	2026-05-02	
3	S003	103	15000	2026-05-03	
4	S004	104	2500	2026-05-04	
5	S001	101	75000	2026-05-01	
6	S002	102	2400	2026-05-02	
7	S003	103	15000	2026-05-03	
8	S004	104	2500	2026-05-04	
9	S001	101	75000	2026-05-01	
10	S002	102	2400	2026-05-02	
11	S003	103	15000	2026-05-03	

Expected Output

Rows returned instantly with lower execution time.

Apply Early Filtering:

OLD QUERY:

```
SELECT *  
  
FROM sales;
```

OPTIMIZED QUERY:

```
SELECT *  
  
FROM sales  
  
WHERE sale_date >= '2026-05-01';
```

```
1  SELECT *  
2  FROM sales  
3  WHERE sale_date >= '2026-05-01';
```

✓ 15 sec - Command executed in 15 sec 319 ms by Chetna Chaudhary on 5/19/2026, 10:57:06 PM

> Spark jobs (3 of 3 succeeded)  Resources  Log

 Table  New chart

Table view

	ABC sale_id	ABC customer_id	ABC product_id	ABC quantity	ABC sale_amount	ABC sale_date	ABC region
1	S001	101	P101	1	75000	2026-05-01	West
2	S002	102	P102	2	2400	2026-05-02	West
3	S003	103	P104	1	15000	2026-05-03	North
4	S004	104	P103	1	2500	2026-05-04	West
5	S001	101	P101	1	75000	2026-05-01	West
6	S002	102	P102	2	2400	2026-05-02	West
7	S003	103	P104	1	15000	2026-05-03	North
8	S004	104	P103	1	2500	2026-05-04	West
9	S001	101	P101	1	75000	2026-05-01	West
10	S002	102	P102	2	2400	2026-05-02	West
11	S003	103	P104	1	15000	2026-05-03	North
12	S004	104	P103	1	2500	2026-05-04	West
13	S001	101	P101	1	75000	2026-05-01	West

Expected Output

Smaller filtered dataset returned.

Reduce Joins:

OLD HEAVY JOIN

```
SELECT *  
  
FROM sales s  
  
INNER JOIN customers c  
  
ON s.customer_id = c.customer_id  
  
INNER JOIN products p  
  
ON s.product_id = p.product_id;
```

OPTIMIZED JOIN

```
SELECT  
  
s.sale_id,  
  
s.sale_amount,  
  
p.product_name  
  
FROM sales s  
  
INNER JOIN products p
```

ON s.product_id = p.product_id;

```
1  SELECT
2  s.sale_id,
3  s.sale_amount,
4  p.product_name
5  FROM sales s
6  INNER JOIN products p
7  ON s.product_id = p.product_id;
8
```

✓ 10 sec - Command executed in 9 sec 994 ms by Chetna Chaudhary on 5/19/2026, 11:00:11 PM

> Spark jobs (10 of 10 succeeded) Resources Log

Table

+ New chart

Table view

	ABC sale_id	ABC sale_amount	ABC product_name
1	S001	75000	Laptop
2	S001	75000	Laptop
3	S001	75000	Laptop
4	S001	75000	Laptop
5	S001	75000	Laptop
6	S001	75000	Laptop
7	S001	75000	Laptop
8	S001	75000	Laptop
9	S001	75000	Laptop
10	S001	75000	Laptop

Expected Output

Smaller joined dataset.

Create Partitioned Table

Add New SQL Cell

```
CREATE OR REPLACE TABLE sales_partitioned
```

```
USING DELTA
```

```
PARTITIONED BY (region)
```

```
AS
```

```
SELECT *
```

```
FROM sales;
```

```
1 CREATE OR REPLACE TABLE sales_partitioned
2 USING DELTA
3 PARTITIONED BY (region)
4 AS
5 SELECT *
6 FROM sales;
```

✓ 9 sec - Command executed in 9 sec 544 ms by Chetna Chaudhary on 5/19/2026, 11:01:07 PM

Expected Output

New table created:

sales_partitioned

Create Aggregated Summary Table

Add SQL Cell

```
CREATE OR REPLACE TABLE sales_summary AS
```

```
SELECT
```

region,

SUM(sale_amount) AS total_sales

FROM sales

GROUP BY region;

```
1 CREATE OR REPLACE TABLE sales_summary AS
2 SELECT
3   region,
4   SUM(sale_amount) AS total_sales
5 FROM sales
6 GROUP BY region;
```

✓ 5 sec - Command executed in 5 sec 338 ms by Chetna Chaudhary on 5/19/2026, 11:02:01 PM

Expected Output

Summary table created.

Enable Parallelism in Pipeline

Actions

Click:

- Copy activity
- Settings tab

Set

Setting	Value
Intelligent throughput optimization	Auto
Degree of copy parallelism	4

Expected Output

Pipeline copy activity configured for parallel execution.

The screenshot displays the Microsoft Fabric Dataflow Designer interface. The top navigation bar shows the workspace 'retail_pipeline_optimized'. The main canvas contains a pipeline with four activities: 'Copy data' (copy_customers), 'Copy data' (copy_products), 'Copy data' (copy_sales), and 'Notebook' (retail_sql_optimized_notebook). The 'Settings' tab for the first 'Copy data' activity is expanded, showing the following configuration:

- Intelligent throughput optimization: Auto
- Use custom value: ☐
- Degree of copy parallelism: 4
- Fault tolerance: Skip incompatible rows
- Enable logging: ☐
- Enable staging: ☐

Step 18 — Run Optimized Pipeline

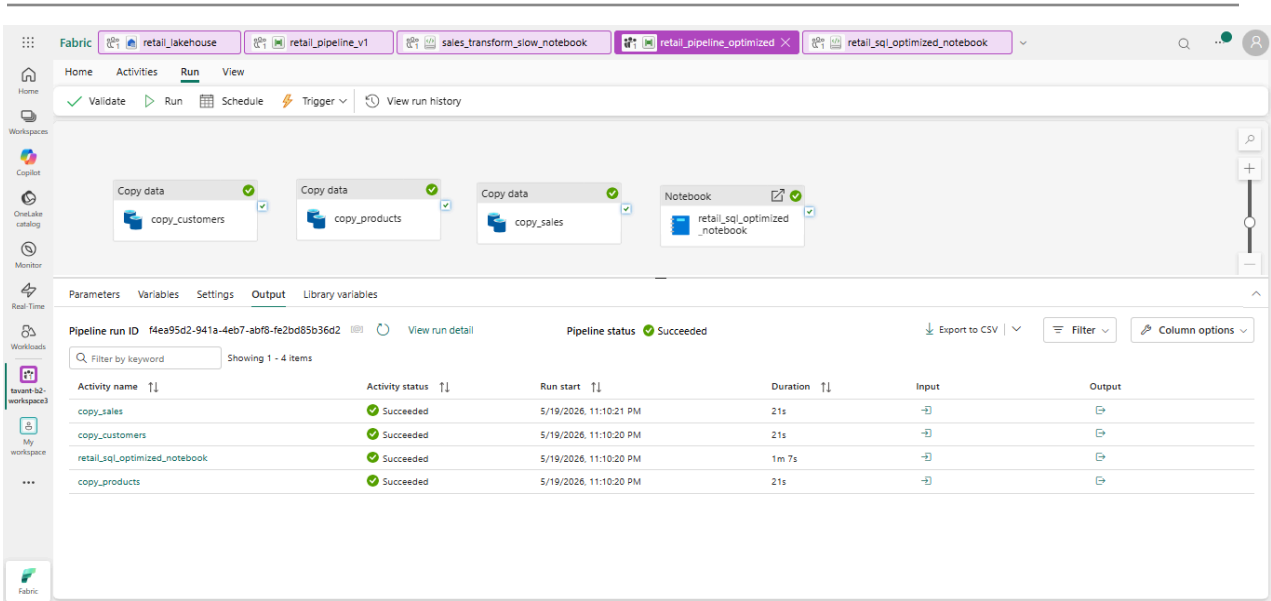
Actions

Run optimized pipeline.

Compare with old pipeline.

Expected Output

Metric	Old	New
Runtime	5 min	2 min
CPU	High	Lower
Throughput	Slow	Faster



Validation

Performance improved successfully.

Step 19 — Governance Policies

Implement Workspace Governance (RBAC)

Actions

Open:

Workspaces → tavant-b2-workspace3

Click:

Manage Access

Add Users / Roles

Assign roles like:

Role	Permission	Governance Purpose
Viewer	Read-only	Controlled data access
Contributor	Edit limited assets	Managed development
Member	Full workspace editing	Team collaboration
Admin	Full control	Governance administration

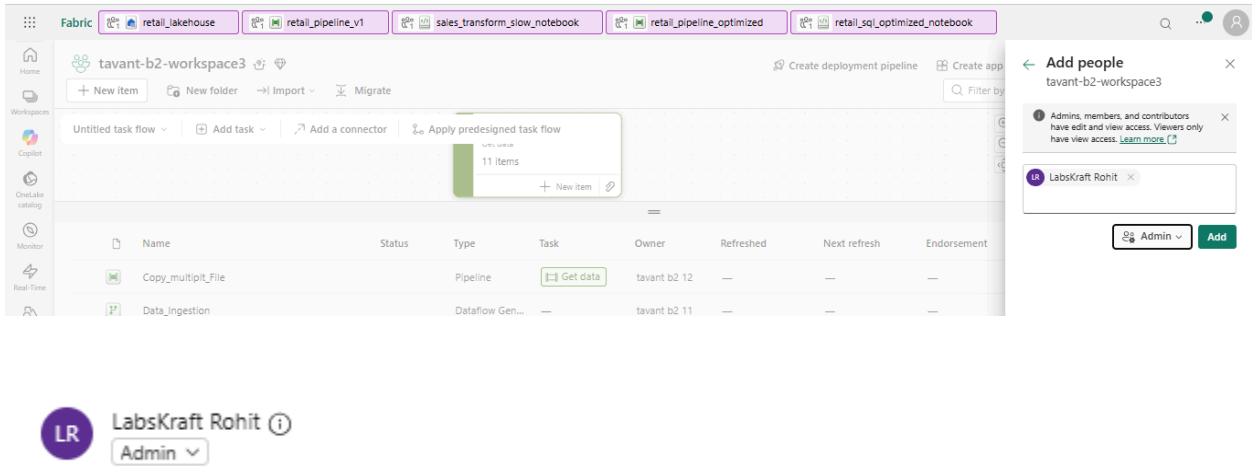
Expected Output

Users appear under workspace access list.

Example:

User	Role
analyst_user	Viewer

engineer_user	Contributor
---------------	-------------



Governance Validation

- Role-based governance implemented
- Least-privilege access enforced

Governance Through Data Quality Validation

Open Notebook

Open:

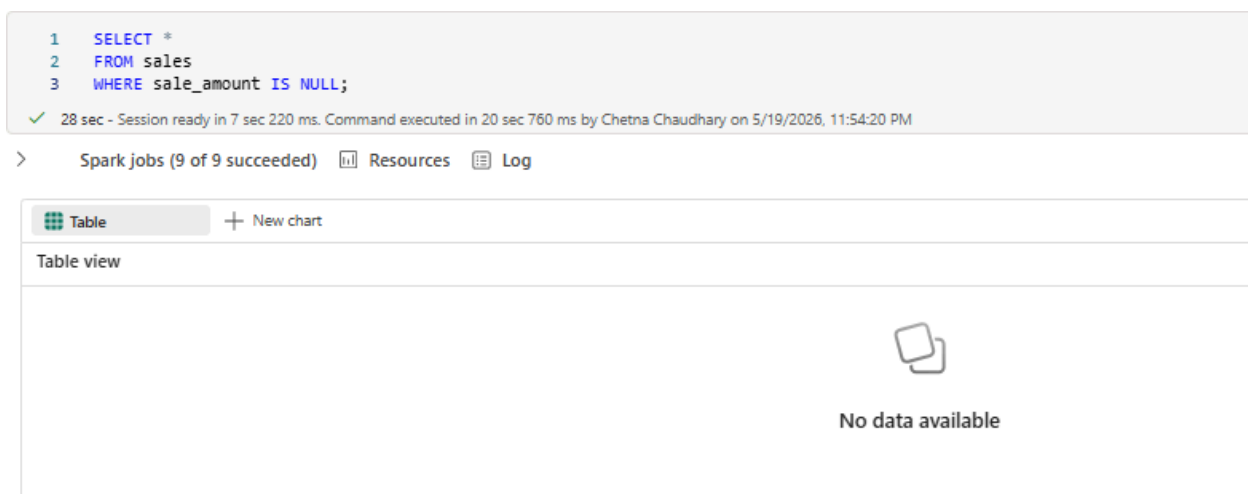
retail_sql_optimized_notebook

Add SQL Validation Cell

```
SELECT *  
  
FROM sales  
  
WHERE sale_amount IS NULL;
```

Expected Output

No NULL rows returned.



The screenshot displays a SQL execution environment. At the top, a code editor shows a query: `1 SELECT *
2 FROM sales
3 WHERE sale_amount IS NULL;`. Below the code, a status bar indicates: `✓ 28 sec - Session ready in 7 sec 220 ms. Command executed in 20 sec 760 ms by Chetna Chaudhary on 5/19/2026, 11:54:20 PM`. Below this, a navigation bar shows `> Spark jobs (9 of 9 succeeded) Resources Log`. The main area is titled `Table` with a `+ New chart` button. Under the `Table view` tab, there is a large empty space with a `No data available` message and a small icon of two overlapping squares.

Governance Validation

- Data quality governance implemented

Governance Through Monitoring & Audit

Actions

From left panel open:

Monitor

Observe

You should see:

- pipeline runs
 - notebook runs
 - copy activity execution
 - success/failure status
 - execution duration
-

Expected Output

Activity	Status
copy_sales	Succeeded
notebook	Succeeded

Monitor
View and track the status of the activities across all the workspaces for which you have permissions within Microsoft Fabric.

Refresh Filter Column Options

Clear all To apply filters, select the values from the Filter dropdown menu.

Activity name	Status	Item type	Start time	Submitted by	Location
retail_sql_optimized_notebook_3f3ad374-9566-...	In progress	Notebook	05/19/2026, 11:53 PM	chetna chaudhary	tavant-b2-workspace3
retail_sql_optimized_notebook_8a50e342-79db-...	Stopped	Notebook	05/19/2026, 11:31 PM	chetna chaudhary	tavant-b2-workspace3
retail_sql_optimized_notebook_86476713-f4c-4...	Succeeded	Notebook	05/19/2026, 11:10 PM	chetna chaudhary	tavant-b2-workspace3
retail_pipeline_optimized	Succeeded	Pipeline	05/19/2026, 11:10 PM	chetna chaudhary	tavant-b2-workspace3
sales_transform_slow_notebook_92f6a590-d4bc-...	Succeeded	Notebook	05/19/2026, 11:04 PM	chetna chaudhary	tavant-b2-workspace3
retail_pipeline_v1	Succeeded	Pipeline	05/19/2026, 11:04 PM	chetna chaudhary	tavant-b2-workspace3
retail_sql_optimized_notebook_d171f0f6-76d5-4...	Stopped	Notebook	05/19/2026, 10:53 PM	chetna chaudhary	tavant-b2-workspace3
Notebook_6_46a33da7-e2b3-49f5-b924-775962...	Succeeded	Notebook	05/19/2026, 10:34 PM	chetna chaudhary	tavant-b2-workspace3
retail_pipeline_v1	Succeeded	Pipeline	05/19/2026, 10:33 PM	chetna chaudhary	tavant-b2-workspace3
Notebook_6_7cd334ef-e466-4d94-af07-1eed915...	Stopped	Notebook	05/19/2026, 10:26 PM	chetna chaudhary	tavant-b2-workspace3
Notebook_3_96b0efb8-165b-4c58-acc4-b51265...	Stopped	Notebook	05/19/2026, 10:04 PM	chetna chaudhary	tavant-b2-workspace3
retail_pipeline_v1	Succeeded	Pipeline	05/19/2026, 9:56 PM	chetna chaudhary	tavant-b2-workspace3

Governance Validation

- Audit monitoring enabled
- Operational governance implemented

Step 20 — Create Warehouse

Actions

Inside:

tavant-b2-workspace3

Click:

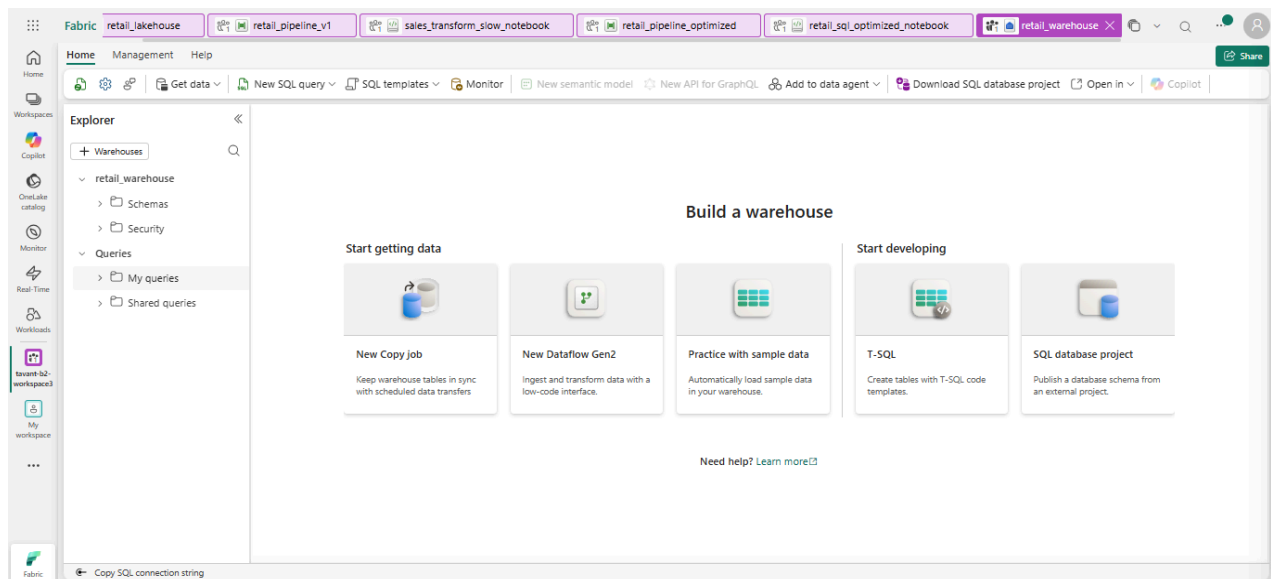
New Item → Warehouse

Name:

retail_warehouse

Expected Output

Warehouse SQL editor opens.



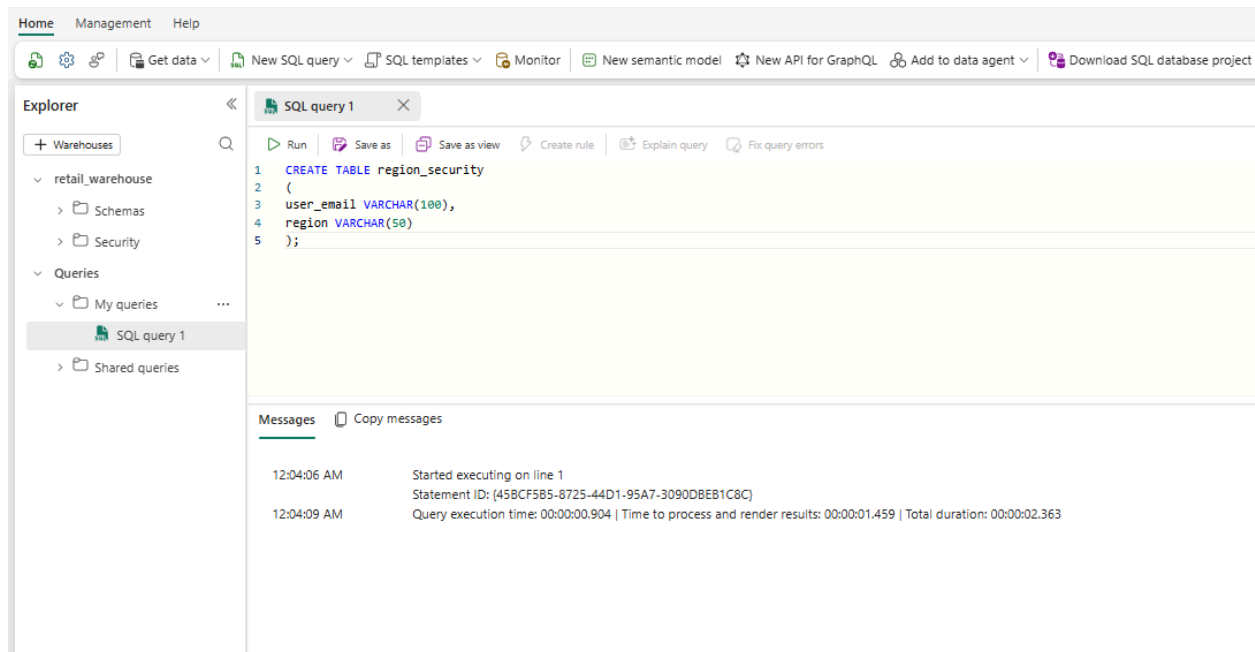
Step 21 — Create Security Table

Run:

```
CREATE TABLE region_security
```

```
(
```

```
user_email VARCHAR(100),  
  
region VARCHAR(50)  
  
);
```



Expected Output

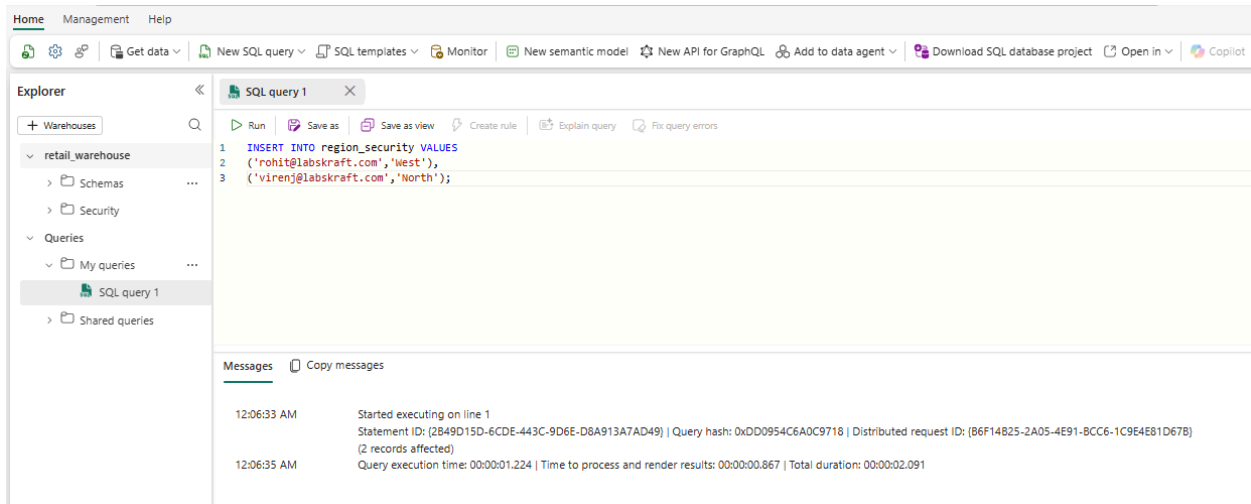
Command completed successfully

Step 22 — Insert Security Data

```
INSERT INTO region_security VALUES  
(  
'west_user@company.com','West'),  
(  
'north_user@company.com','North');
```

Expected Output

Rows inserted successfully.



Step 23 — Create RLS Policy

Run:

```
CREATE FUNCTION dbo.fn_region_access(@region AS VARCHAR(50))
```

```
RETURNS TABLE
```

```
WITH SCHEMABINDING
```

```
AS
```

```
RETURN
```

```
SELECT 1 AS access_result
```

```
WHERE @region IN
```

```
(
```

```
SELECT region  
  
FROM dbo.region_security  
  
WHERE user_email = USER_NAME()  
  
);
```



```
SQL query 1  
Run Save as Save as view Create rule Explain query Fix query errors  
1 CREATE FUNCTION dbo.fn_region_access(@region AS VARCHAR(50))  
2 RETURNS TABLE  
3 WITH SCHEMABINDING  
4 AS  
5 RETURN  
6 SELECT 1 AS access_result  
7 WHERE @region IN  
8 (  
9     SELECT region  
10    FROM dbo.region_security  
11   WHERE user_email = USER_NAME()  
12 );
```

Messages Copy messages

12:09:57 AM Started executing on line 1
12:09:57 AM Query execution time: 00:00:00.010 | Time to process and render results: 00:00:00.773 | Total duration: 00:00:00.783

Expected Output

Function created successfully.

Step 24 — Load data from lakehouse to warehouse

Use Data Pipeline

Open:

retail_pipeline_optimized

Add Copy Activity

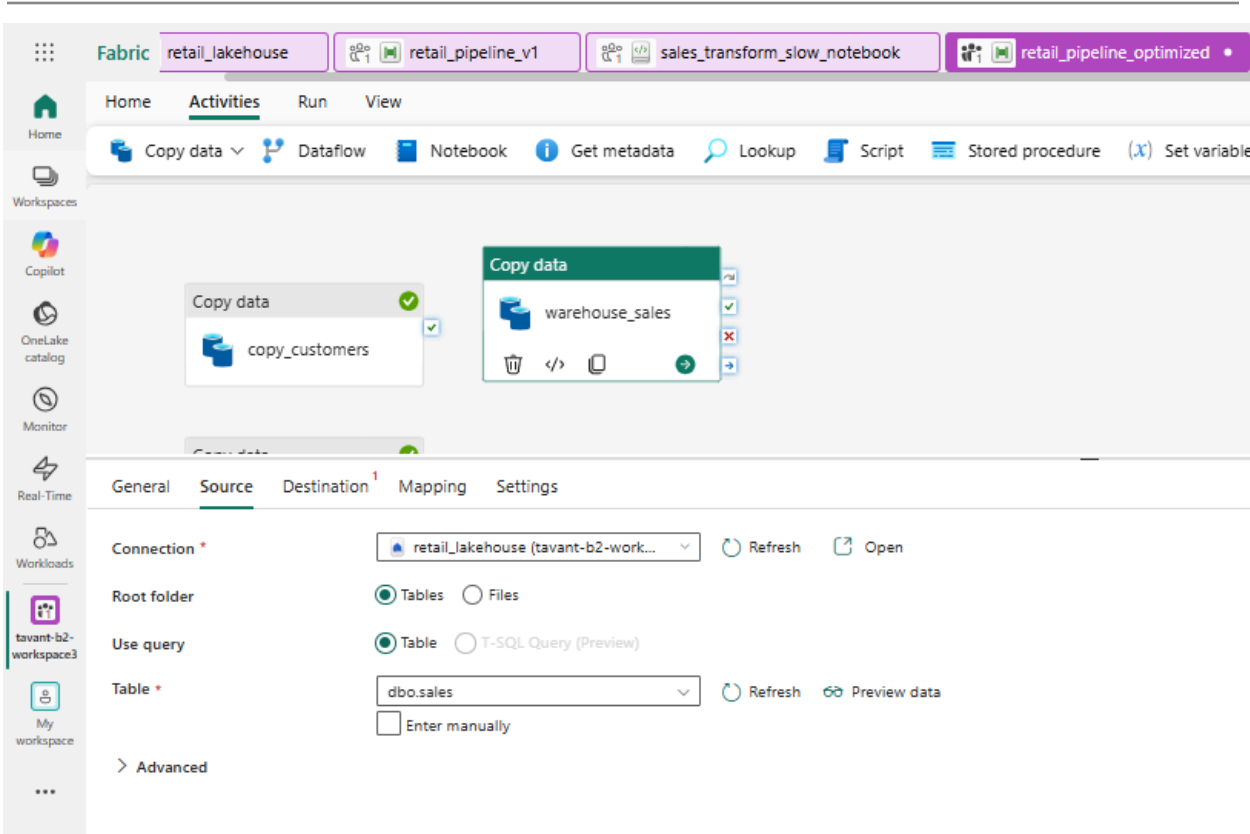
Drag:

Copy Data

onto pipeline.

Source Configuration

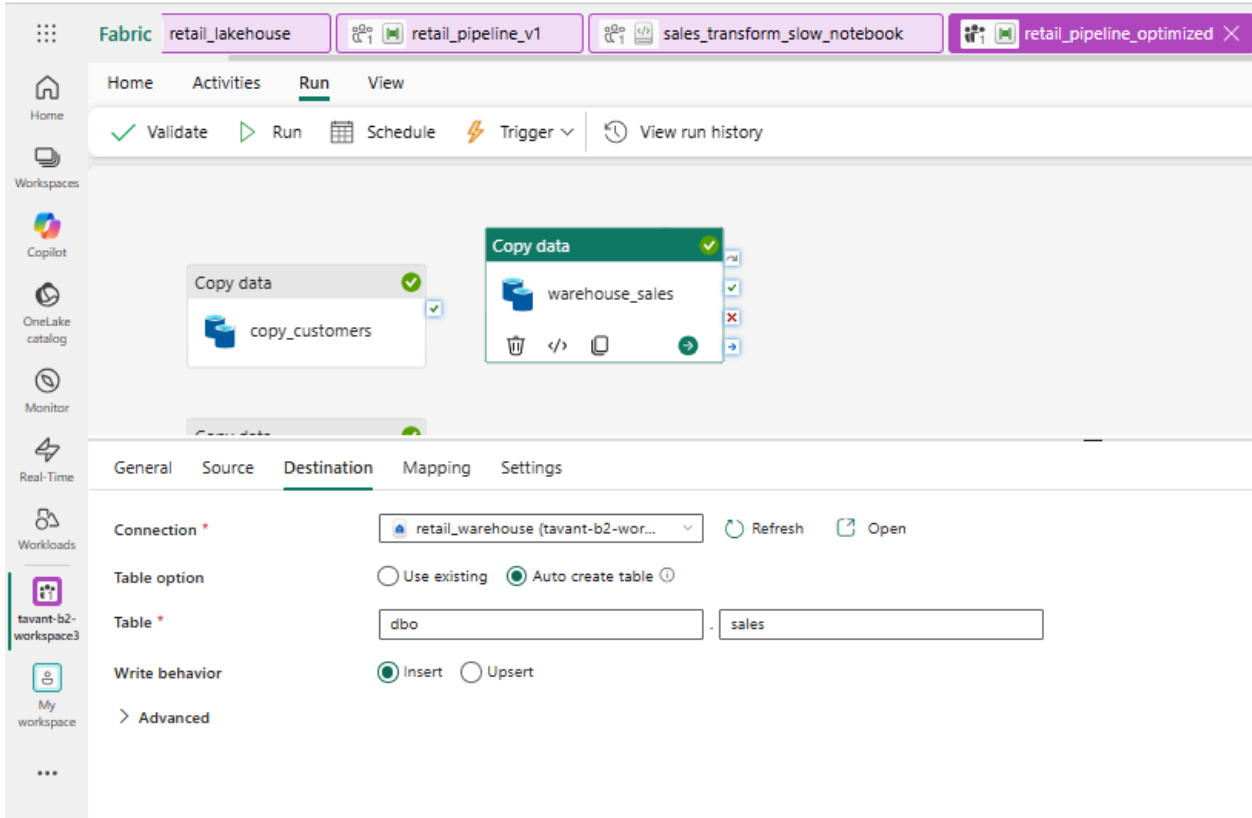
Field	Value
Source Type	Lakehouse Table
Connection	retail_lakehouse
Table	sales



Destination Configuration

Field	Value
Destination Type	Warehouse
Warehouse	retail_warehouse
Schema	dbo

Table	sales
-------	-------



Expected Output

Pipeline copies Lakehouse data into Warehouse table.

Run Pipeline

Click:

Run

Expected Output

Copy activity succeeds.

The screenshot shows the 'Run' tab of a pipeline named 'retail_pipeline_v1' in Microsoft Fabric. The pipeline status is 'Succeeded'. Below the pipeline diagram, the 'Output' tab displays a table of activity results.

Activity name	Activity status	Run start	Duration	Input	Output
copy_sales	Succeeded	5/20/2026, 12:31:16 AM	20s		
warehouse_sales	Succeeded	5/20/2026, 12:31:16 AM	44s		
copy_products	Succeeded	5/20/2026, 12:31:16 AM	18s		
retail_sql_optimized_notebook	Succeeded	5/20/2026, 12:31:16 AM	1m 22s		
copy_customers	Succeeded	5/20/2026, 12:31:16 AM	19s		

Validate Warehouse Data

Inside Warehouse run:

SELECT * FROM dbo.sales;

The screenshot shows the 'SQL query 2' results in Microsoft Fabric. The query executed is 'SELECT * FROM dbo.sales;'. The results are displayed in a table with 7 columns and 7 rows.

ABC sale_id	ABC customer_id	ABC product_id	ABC quantity	ABC sale_amount	ABC sale_date	ABC region
1 S001	101	P101	1	75000	2026-05-01	West
2 S002	102	P102	2	2400	2026-05-02	West
3 S003	103	P104	1	15000	2026-05-03	North
4 S004	104	P103	1	2500	2026-05-04	West
5 S001	101	P101	1	75000	2026-05-01	West
6 S002	102	P102	2	2400	2026-05-02	West
7 S003	103	P104	1	15000	2026-05-03	North

Expected Output

Sales rows visible successfully.

Step 25 — Apply Security Policy

```
CREATE SECURITY POLICY region_policy
```

```
ADD FILTER PREDICATE dbo.fn_region_access(region)
```

```
ON dbo.sales
```

```
WITH (STATE = ON);
```

The screenshot shows a SQL query editor window titled "SQL query 2". The query text is as follows:

```
1 CREATE SECURITY POLICY region_policy
2 ADD FILTER PREDICATE dbo.fn_region_access(region)
3 ON dbo.sales
4 WITH (STATE = ON);
```

Below the query editor, there is a "Messages" section with a "Copy messages" button. The messages indicate successful execution:

```
12:36:24 AM      Started executing on line 1
12:36:25 AM      Query execution time: 00:00:00.018 | Time to process and render results: 00:00:00.765 | Total duration: 00:00:00.783
```

Expected Output

Security policy active.

Step 26 — Validate Lineage

Open Lineage View

Actions

Workspace → Lineage

Expected Output

Visual dependency graph:

CSV Files

↓

Lakehouse

↓

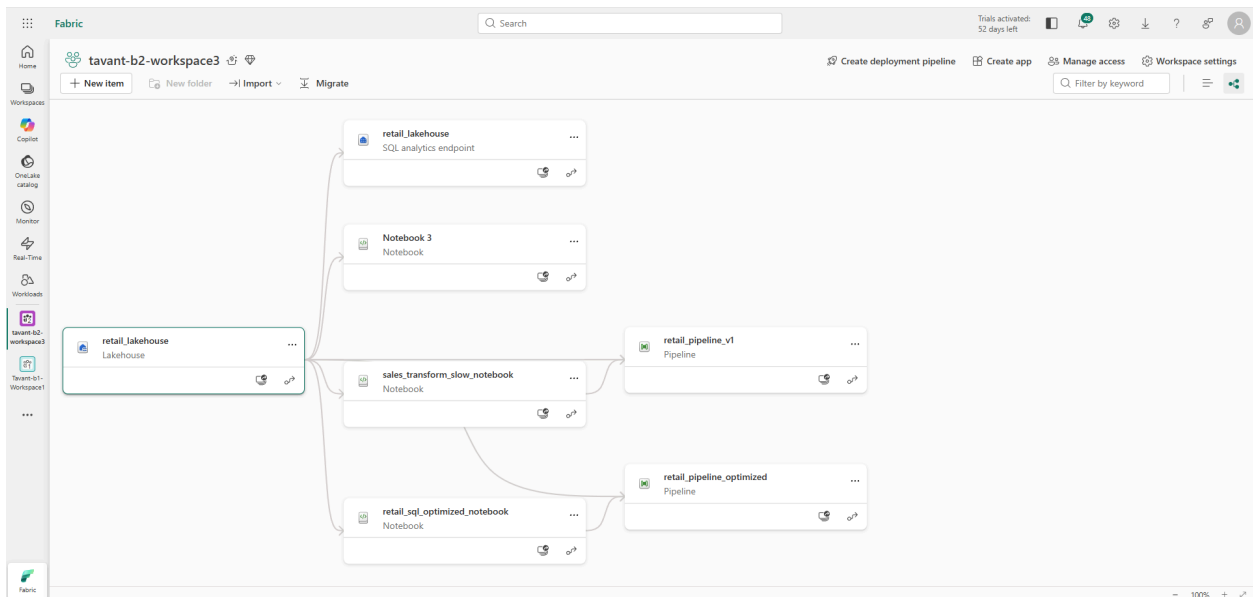
Pipeline

↓

Warehouse

↓

Reports



Step 27 — Monitor Metrics

Actions

Open:

Monitor Hub

Track:

- duration
- failures
- retries
- throughput

FINAL VALIDATION

Expected Final Assets

Asset	Expected
Lakehouse	Created
Tables	customers/products/sales
Pipelines	Slow + Optimized
Governance	Applied
Warehouse	Created
RLS	Working
Lineage	Visible
Monitoring	Active